

Git and GitHub

Version Control System:

Version Control System is also known as Software Configuration Management (SCM)
OR

Source Code Management (SCM) System.

- **Need of Version Control System:**

Being a developer we have to write several files which contain source code.

Developer => Write Code => Files

- ❖ **Client project**

- 100 files developed
- client suggested some changes
- changed some files source code to meet client requirement
- I gave the demo and client suggested some more changes
- I changed some files source code to meet client requirement
- I gave demo 3rd time
- Client asked for first version only
- What should I do ????

- **We should not overwrite our code.**
- **Every version we have to maintain**

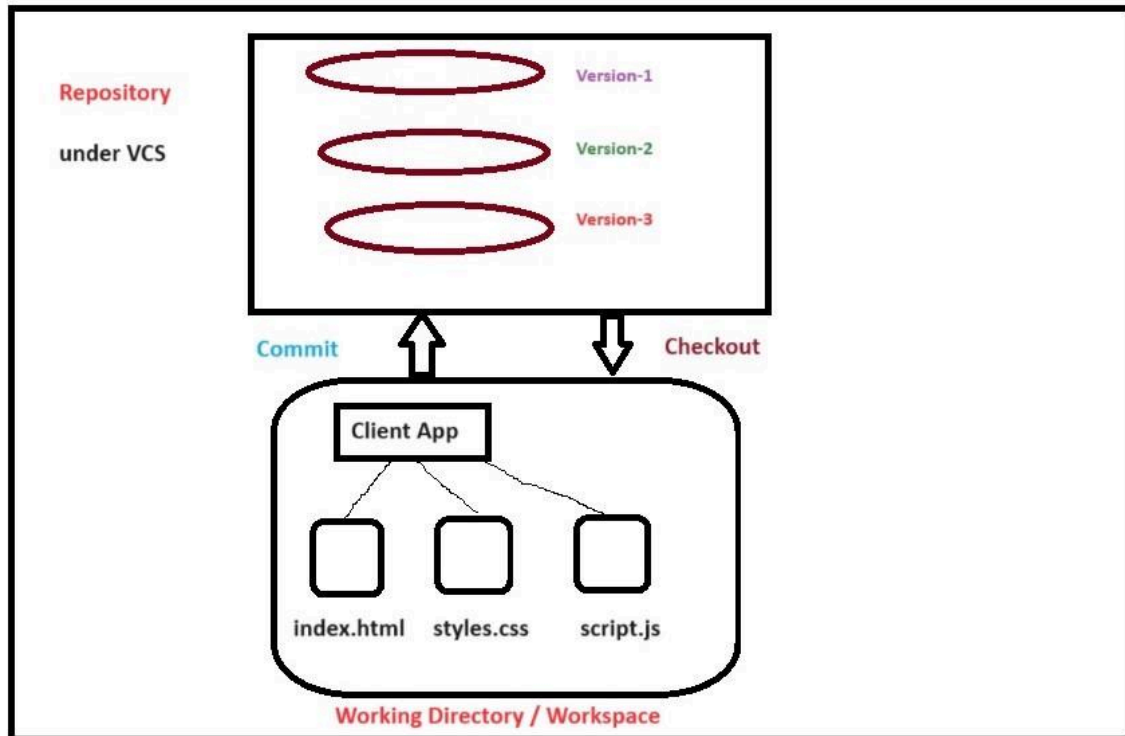
1. Maintaining multiple versions manually is a very complex activity.
2. Dev-A and Dev-B working on the code. At last we have to merge the code developed by both developers and we have to deliver it to the client. If both developers developed a file named with index.html, then one copy will be overwritten with another copy, which creates abnormal behaviour. We should not overwrite our code.
3. Every change should be tracked like

- a. Who did the change
 - b. When he did the change
 - c. Which changes he did, etc
- and all changes should be maintained

- 4. Overwriting of the code should not happen.
- 5. Developers have to share their code to peer developers, so that multiple developers will work in a collaborative way.
- 6. Parallel development must be required.

- **How Version Control System Works:**

- The version control system always talks about files which contain source code.
- Everyone required a version control system to maintain different versions of their document.
 - **Tester** To maintain different versions of test script
 - **Architect** To maintain different versions of Documents
 - **Project Manager** To maintain different versions of Excel sheets,etc



- **Basic Terminology of Version Control System:**

1. **Working Directory:**

- Where developers are required to create/modify files.
- Here version control is not applicable. Here we won't use the work like version-1, version-2 etc.

2. **Repository:**

- Where we have to store files and metadata.
- Here version control is applicable.
- Here we can talk about versions like version-1, version-2 etc

3. **Commit:**

- The process of sending files from working directory to the repository

4. **Checkout:**

- The process of sending files from repository to working directory.

- **Benefits of Version Control System:**

1. We can maintain different versions and we can choose any version based on client requirements.
2. With every version/commit we can maintain metadata like:
 - commit message
 - who did changes
 - when he did the change
 - what changes he did
3. Developers can share the code to the peer developers in a very easy way.
4. Multiple developers can work in a collaborative way without interfering with each other's work.
5. Parallel development. i.e developers can work on different features or fixes in parallel, and the VCS merges their changes intelligently.
6. We can provide access control like
 - who can read code
 - who can modify code
7. Serves as a backup mechanism for code. Even if a developer's local machine crashes or a server goes down, the code can be recovered from the central repository.

● **Types of Version Control Systems:**

There are 2 types of VCSs:

1. Centralized Version Control System
2. De Centralized/Distributed Version Control System

Centralized Version Control System:

- The name itself indicates that this type contains only one central repository and every developer should be connected to that repository.
- The total project code will be stored in the central repository. If 4 developers are there, still we have only one repository.

Example: CVS, SVN, Perforce, TFS, Clearcase etc.

Advantages:

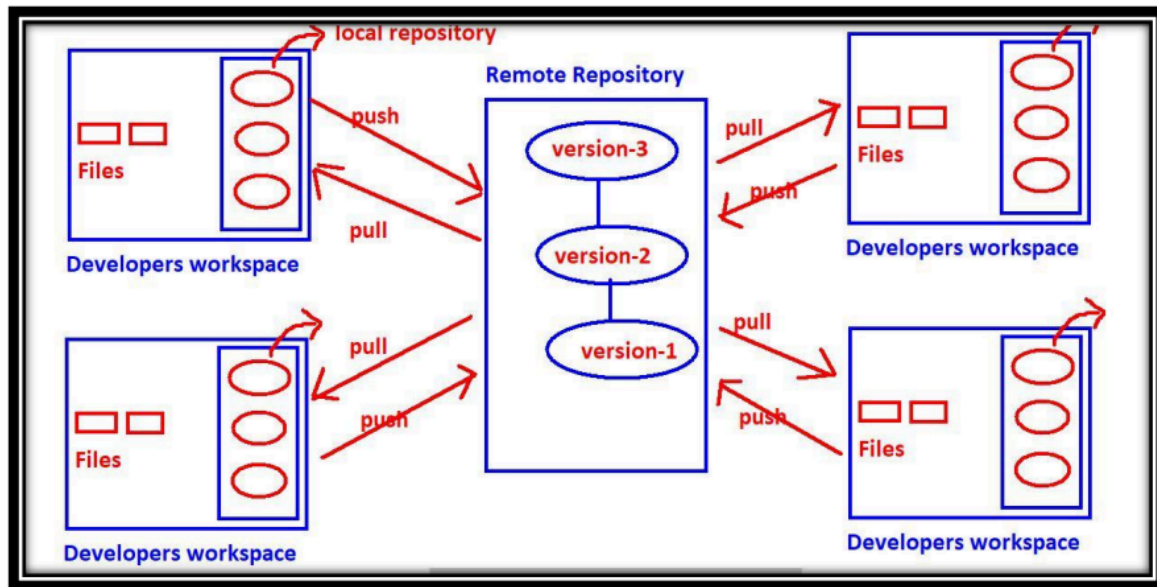
- This type of VCS is very easy to set up and use.
- Easier to manage access control and permissions
- Simpler to understand and use, especially for users who are habituated to a linear workflow.

Disadvantages:

- Central Repository is the only place where everything is stored, which causes a single point of failure. If something goes wrong to the central repository then recovery is very difficult
- All commit and checkout operations should be performed by connecting to the central repository via network. If network outage, then no version control to the developer. i.e in this type, developer work space and remote repository server should be connected always.
- All commit and checkout operations should be performed by connecting to the central repository via network and hence these operations will become slow, which causes performance issues. No local operations and every version control operation should be remote operation
- The organisation of the central repository is very complex if the number of developers and files increases. Etc.

Distributed Version Control Systems:

- The name itself indicates the repository is distributed and every developer's workspace contains a local copy of the repository. There is no question of a central repository.
- If 4 developers are there then 4 local repositories will be there for each developer system.



- The checkout and commit operations will be performed locally. Hence performance is more.
- To perform checkout and commit operations, the network is not required. Hence if there is any network outage, still version control is applicable.
- If something goes wrong in any repository there is a chance to recover. There is no question of a single point of failure.
- To perform push and pull operations the network must be required, but these operations are not the most common operations and we are performing very rarely.
- Every developer has his own local copy of the repository. It is not centralized and it is distributed only.
- The main job of remote repositories is just to share our work to peer developers.
- High availability, speed and there is no single point of failure are the main reasons for popularity of this model.

Example: Git, Mercurial, Fossil.

Note: **commit** and **checkout** operations will be performed between workspace and repository.

Workspace => commit => Repository
Repository => checkout => Workspace

Whereas push and pull operations will be performed between local repositories to the Remote repository.

Local-Repository => push => Remote-Repository
Remote-Repository => pull => Local-Repository

What is Git ?

- Git is Distributed Version Control System Software.
- Git is not an acronym and hence no expansion. But most of the people abbreviate it as **Global Information Tracker**.
- GIT is developed by Linus Torvalds(Finnish software engineer), who also developed Linux Kernel.
- Most of the companies like Microsoft, Facebook, Yahoo, LinkedIn, Intel using Git as Version Control System Tool.

Features of GIT:

- Git is very popular because of the following features

1. **Distributed:**

Git is developed based on Distributed Version Control System Architecture. Because of Distributed Architecture it has several advantages:

- a. Every Developer has his own local repository. All the operations can be performed locally. Hence local repo and remote repo need not be connected always.
- b. All operations will be performed locally, and hence performance is high when compared with other VCSs. i.e it is very speedy.
- c. Most of the operations are local. Hence we can work offline most of the time.
- d. There is no single point failure as Every Developer has his own local repository.
- e. It enables parallel development & automatic-backups.

2. Staging Area:

- It is also known as index area. There is logical layer/virtual layer in git between working directory and local repository.

WorkingDirectory => Staging Area => Local Repository

- We cannot commit the files of the working directory directly. First we have to add to the staging area and then we have to commit.
- This staging area is helpful to double check/cross-check our changes before commit.
- Git stores files in the repository in some hash form, which saves space.
- GIT will use an internal snapshot mechanism for this. All these conversions and taking snapshots of our data will happen in the staging area before commit.

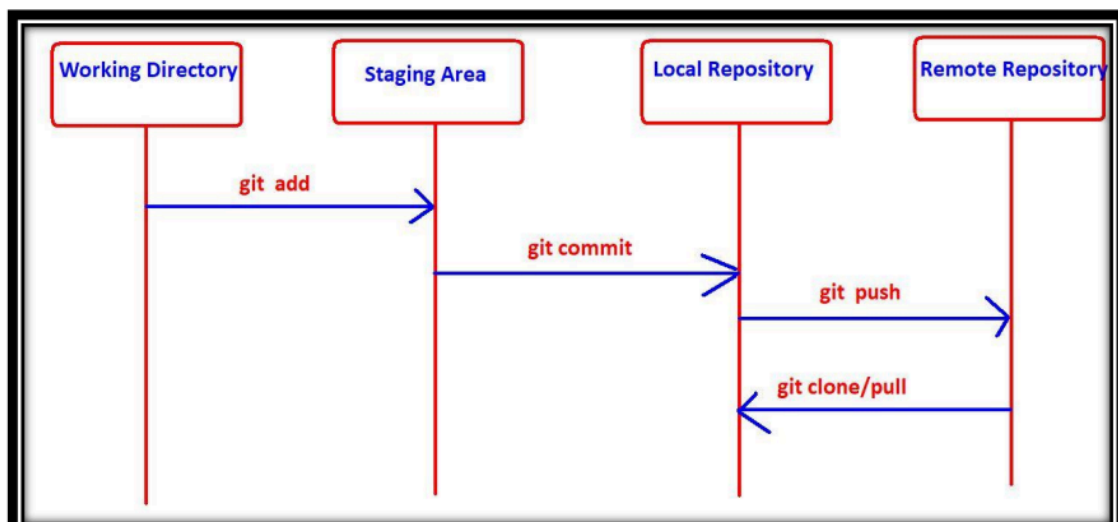
3. Branching and Merging:

- We can create and work on multiple branches simultaneously and all these branches are isolated from each other. It enables multiple work flows.
- The default workflow starts from the **main** or **master** branch.
- We can merge multiple branches into a single branch. We can commit branch wise also.

The **main** or **master** branch:

- In Git, the "main" or "master" branch is the default branch that is created when you initialize a new repository. It serves as the primary line of development and typically represents the stable, production-ready version of your project. Historically, Git repositories commonly used the term "master" for the main branch. However, in recent years, there has been a movement within the Git community to replace "master" with more inclusive terms like "main" to avoid potentially offensive connotations associated with the term "master."

Architecture of GIT:



Git contains 2 types of repositories:

1. Local Repository:

2. Remote Repository:

1. Local Repository:

- For every developer, a separate local repository is available. Developers can perform all checkout and commit operations wrt local repository only.
- To perform a commit operation, first he has to add files to the staging area by using the git add command, and then he has to commit those changes to the local repository by using git commit command. Hence commit in GIT is a 2-step process.
- commit is applicable only for staging area files but not for working directory files.

2. Remote Repository:

- The remote repository is a version of your project's repository that is hosted on a remote server, such as **GitHub** or **Bitbucket**. It serves as a centralized location for collaboration, allowing multiple developers to work on the same project and share their changes.
- If the developer wants to share his work to the peer developers then he has to push his local repository to the remote repository by using **git push** command.
- Remote repository contains total project code, which can be accessible by all developers.
- New developers can get local repositories by cloning remote repository. For this we have to use the git **clone** command.
- A developer can get updates from the remote repository to the local repository by using the git **pull** command.

Git vs GitHub:

Git:

- ☐ Git is a distributed version control system (DVCS) used for tracking changes in source code during software development.
- ☐ It allows multiple developers to collaborate on a project by managing different versions of files and tracking changes made by each contributor.
- ☐ Git operates locally on a developer's machine, allowing them to work offline and commit changes to their local repository.
- ☐ It provides features like branching, merging, and version history tracking.

GitHub:

- ☐ GitHub is a web-based platform and hosting service for Git repositories.
- ☐ GitHub is the most popular service provider of remote repositories.
- ☐ It provides a centralized location for hosting Git repositories and collaborating on projects.
- ☐ Developers can create repositories on GitHub to store their code, collaborate with others, and manage project issues and tasks.
- ☐ GitHub offers additional features like pull-requests, code-reviews, project-management tools, and integration with other services.
- ☐ While Git is the version control system, GitHub acts as a hosting service and collaboration platform built around Git.

In summary, Git is the version control system used for tracking changes in source code, while GitHub is a web-based platform that provides hosting for Git repositories and facilitates collaboration among developers. Many other platforms like **GitLab** and **Bitbucket** also offer similar features to GitHub but may have different offerings and pricing models.

Life-cycle of File in GIT:

Every file in GIT is in one of the following states:

1. Untracked:

- The files which are newly created in the working directory and git is not aware of these files are said to be in untracked state.

2. Staged:

- The files which are added to the staging area are said to be in staged state.

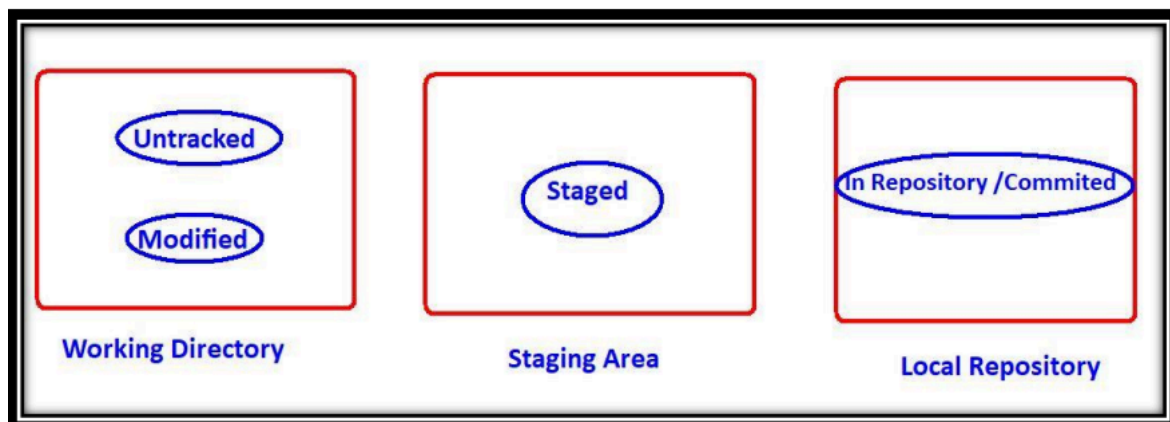
- These files are ready for commit.

3. In Repository/ Committed:

- Any file which is committed is said to be In Repository/Committed State.

4. Modified:

- Any file which is already tracked by git, but it is modified in the working directory is said to be in the Modified State.



Git Installation:

- Download it from <https://git-scm.com/download/win> for the appropriate operating system. and install it with default options.
- Open git bash/CMD/Terminal in VS Code & check the git version using following command

git --version

Tips:

To exit the editor that sometimes appear to write the commit messages etc. type **:wq** then press enter

Sometimes complete output does not appear on terminal or it is stuck as it takes too long to print the output; to skip the output printing and see prompt again to type; press **~, Q** [i.e. **press ~ then Q**].

Use **cls** command on the window terminal and **clear** command in git bash to clear the output from terminal and make it blank.

- If we just type **git**, on the terminal then we will get complete options available with the git command.

Commonly used commands in the GIT:

1. git config

To start working with Git, we should configure user name and mail id so that git can use this information in the commit records. We can perform these configurations with the following commands:

```
git config --global user.name "Ratan"  
git config --global user.email "ratankapr22@gmail.com"
```

Note: global means these configurations are applicable for all repositories created by git. If we are not using global then it is applicable only for the current repository.

git config --list

To list out all git configurations

git config user.name

To display user name

git config user.email

To display user email

We can change user name and mail id with the same commands

```
git config --global user.email "ratan@gmail.com"  
git config --global user.name "ratan"
```

Understanding Working Directory, Staging Area and Local Repository

1. Creating workspace (create a folder in the local computer)
 2. git initialization (creating the local repository)
 3. Creating files with some content in the working directory
 4. Adding these files to staging area
 5. Commit those changes to local repository
- Create a new folder(workspace) **git-practice** on the Desktop and open the vs-code and open this folder.
 - Open the terminal/powershell/git-bash inside the vs-code. Make sure we are inside the **git-practice** folder.

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/git-practice
```

Now **git-practice** acts as a working directory. We have to request git, to provide version control for this directory. For this we have to use the git **init** command.

2. git init

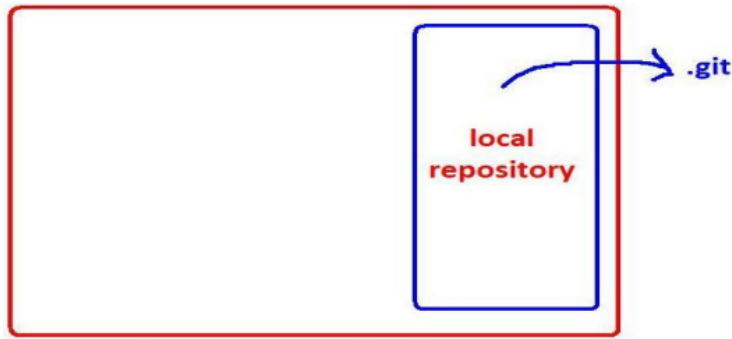
This command will provide empty repository for our working directory, so that

version control is applicable for our workspace.

The name of the empty directory is **.git**, which is hidden directory

This **.git** file contains the metadata for our local repository.

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/git-practice
$ git init
Initialized empty Git repository in
C:/Users/ratan/OneDrive/Desktop/git-practice/.git/
```



git-practice => working directory

Note:

1. If our working directory contains any files, then these files won't be added to the local repository by default, we have to add them explicitly.
2. If our working directory already contains local repository(.git), still if we call git init command, then there is no impact

Creating Files with some Content and adding to staging Area

Create a file **index.html** that has some HTML code

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>Document</title>
</head>

<body>

  <h1>This is a simple HTML file</h1>

</body>

</html>
```

3. git status

Write this command to check files in the working directory & staging area , like which files are untracked, which are modified, which are staged etc.; observe the output carefully.

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/git-practice (master)
$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        index.html

nothing added to commit but untracked files present (use "git add" to track)
```

4. git add

To add files from working directory to staging area for tracking/committing purpose, we have to use the git add command.

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/git-practice (master)
$ git add index.html
```

Note: To add multiple files inside the staging area we can make use of the following command
`git add .`

Again write **git status** command to check file in working directory & staging area; observe the output carefully.

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/git-practice (master)
$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   index.html
```

5. git commit

If we want to commit staged changes, then we have to use the **git commit** command.

For every commit, a unique commit id will be generated. It is of 40-length hexadecimal string.

This unique id is considered a hash, which is generated based on the content of files.

While using git commit command, commit message is mandatory.

Note: For every commit, git records author name, mail id, timestamp and commit message.

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/git-practice (master)
```

```
$ git commit -m "index.html file is created and committed"
[master (root-commit) 427f56f] index.html file is created and
committed
1 file changed, 13 insertions(+)
create mode 100644 index.html
```

Again write **git status** command to check file in working directory & staging area; observe the output carefully.

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/git-practice (master)
$ git status
On branch master
nothing to commit, working tree clean
```

6. git log

This command is used to see the all commit history. Observe the output very carefully.

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/git-practice (master)
$ git log
commit 427f56f0fbab2c2d58953c3e56d6aed97e212da5 (HEAD -> master)
Author: Ratan <ratankapr22@gmail.com>
Date: Sat May 25 18:05:31 2024 +0530

    index.html file is created and committed
```

Now Create a file **styles.css**, add following code to this file

```
body{
    margin: 0;
    padding: 0;
    background-color: beige;
```

```
}
```

Now add following code in the **index.html** file

```
<link rel="stylesheet" href="styles.css" />
```

Again write **git status** command to check file in working directory & staging area; observe the output carefully.

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/git-practice (master)
$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working
directory)
        modified:   index.html

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        styles.css

no changes added to commit (use "git add" and/or "git commit -a")
```

Write **git add** command to add file style.css to staging area

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/git-practice (master)
$ git add styles.css
```

Again write **git status** command to check file in working directory & staging area; observe the output carefully.

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/git-practice (master)
```

```
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   styles.css

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   index.html
```

Use the **git restore** command to restore the modified file to its initial state/last committed state.

Example:

git restore index.html

Write **git commit** command to commit the file **style.css** to local git repo with the message "style.css file is committed"

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/git-practice (master)
$ git commit -m "styles.css file is committed"
[master 09d2659] styles.css file is committed
1 file changed, 5 insertions(+)
create mode 100644 styles.css
```

Write **git log** command to see commit history; observe the output very carefully

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/git-practice (master)
$ git log
commit 09d265988802b4677475bb4e7a5be2a474eabed1 (HEAD -> master)
Author: Ratan <ratankapr22@gmail.com>
Date: Sat May 25 18:43:44 2024 +0530

    styles.css file is committed

commit 427f56f0fbab2c2d58953c3e56d6aed97e212da5
Author: Ratan <ratankapr22@gmail.com>
```

Date: Sat May 25 18:05:31 2024 +0530

index.html file is created and committed

Again write **git status** command to check file in working directory & staging area; observe the output carefully.

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/git-practice (master)
$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working
directory)
        modified:   index.html

no changes added to commit (use "git add" and/or "git commit -a")
```

Write **git add** command to add all files from working directory to staging area [Yet this time we have only one file but command should be capable of adding all files from working directory]

git add .

Write **git commit** command to commit the file index.html to local git repo with the message "index.html file is edited"

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/git-practice (master)
$ git commit -m "index.html file is edited"
[master f275be1] index.html file is edited
1 file changed, 8 insertions(+), 2 deletions(-)
```

Write **git log** command to see commit history; observe the output very carefully

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/git-practice (master)
$ git log
```

```
commit f275be1d61bcd0a4527b722a4355da346b625cf3 (HEAD -> master)
Author: Ratan <ratankapr22@gmail.com>
Date: Sat May 25 18:50:19 2024 +0530
```

index.html file is edited

```
commit 09d265988802b4677475bb4e7a5be2a474eabed1
Author: Ratan <ratankapr22@gmail.com>
Date: Sat May 25 18:43:44 2024 +0530
```

styles.css file is committed

```
commit 427f56f0fbab2c2d58953c3e56d6aed97e212da5
Author: Ratan <ratankapr22@gmail.com>
Date: Sat May 25 18:05:31 2024 +0530
```

index.html file is created and committed

Note: HEAD = Pointer to current commit.

Write **git log --oneline** command to see commit history in brief i.e. every commit details should be in just one line; observe the output very carefully

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/git-practice (master)
$ git log --oneline
f275be1 (HEAD -> master) index.html file is edited
09d2659 styles.css file is committed
427f56f index.html file is created and committed
```

Getting log information of a particular file:

Syntax:

git log <filename>

git log --oneline <filename>

Example:

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/git-practice (master)
$ git log index.html
commit f275be1d61bcd0a4527b722a4355da346b625cf3 (HEAD -> master)
Author: Ratan <ratankapr22@gmail.com>
Date: Sat May 25 18:50:19 2024 +0530

    index.html file is edited

commit 427f56f0fbab2c2d58953c3e56d6aed97e212da5
Author: Ratan <ratankapr22@gmail.com>
Date: Sat May 25 18:05:31 2024 +0530

    index.html file is created and committed
```

Note: There are multiple options available for git log command to see the history.

```
git log --help
```

This command is used to open the **official Git manual page (documentation)** for the `git log` command.

To get the last 2 commit info:

```
git log -2

//Or

git log -2 --oneline
```

To search a commit based on the message:

Here we need to make use of the **--grep** flag.

Syntax:

git log --grep="patten"

Example:

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/git-practice (master)
$ git log --grep="edited"
commit f275be1d61bcd0a4527b722a4355da346b625cf3 (HEAD -> master)
Author: Ratan <ratankapr22@gmail.com>
Date: Sat May 25 18:50:19 2024 +0530

    index.html file is edited
```

7. git diff

The **git diff** command is commonly used to find differences between the content of files in various states. The typical comparisons include:

1. Between the working directory and the staging area
2. Between the working directory and the last commit
3. Between the staged area and the last commit
4. Between the working directory and a particular commit
5. Between the staged area and a particular commit
6. Between two specified commits

"Diff" stands for **difference**, and it highlights the changes between the specified file states.

Demo Example:

file1.txt:

First line in file1.txt
Second line in file1.txt

file2.txt:

First line in file2.txt
Second line in file2.txt

First commit: Both files are committed, each containing 2 lines.

Now add 2 more lines in both files. And commit them.

file1.txt

First line in file1.txt
Second line in file1.txt
Third line in file1.txt
Fourth line in file1.txt

file2.txt

First line in file2.txt
Second line in file2.txt
Third line in file2.txt
Fourth line in file2.txt

Second commit: Both files are updated and committed, each containing 4 lines.

Now we are adding new line in file1.txt in working directory

file1.txt

First line in file1.txt
Second line in file1.txt
Third line in file1.txt
Fourth line in file1.txt
Fifth line in file1.txt

Add file1.txt inside the staging area.

```
git add file1.txt
```

Adding Another Line in Working Directory

file1.txt

First line in file1.txt
Second line in file1.txt
Third line in file1.txt
Fourth line in file1.txt
Fifth line in file1.txt
Sixth line in file1.txt

Case1: To see the difference in File Content between Working Directory and staging Area.

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/git-practice (master)
$ git diff file1.txt
diff --git a/file1.txt b/file1.txt
index e05f0a5..78ed7b9 100644
--- a/file1.txt
+++ b/file1.txt
@@ -2,4 +2,5 @@ First line in file1.txt
  Second line in file1.txt
  Third line in file1.txt
  Fourth line in file1.txt
- Fifth line in file1.txt
\ No newline at end of file
+ Fifth line in file1.txt
+ Sixth line in file1.txt
```

Here:

```
diff --git a/file1.txt b/file1.txt
```

The above line indicates the comparison between the two versions of file1.txt.

a/file1.txt: is the version from source (staging area copy).
b/file1.txt: is the version from destination (working directory copy).

--- a/file1.txt

Shows lines from the source version (staging area).

+++ b/file1.txt

Shows lines from the destination version (working directory).

Hunk Header

@@ -2,4 +2,5 @@

-2,4: The changes start at line 2 in the source version (staging area) and affect 4 lines(2,3,4,5).

+2,5: The changes start at line 2 in the destination version (working directory) and affect 5 lines(2,3,4,5,6).

Lines with Prefixes

Lines with a - prefix are present in the source version but not in the destination version.

Lines with a + prefix are present in the destination version but not in the source version.

In this case:

-Fifth line in file1.txt: This line exists in the source (staging area).

+Fifth line in file1.txt: This line also exists in the destination (working directory) but is shown to provide context.

+Sixth line in file1.txt: This line is newly added in the destination (working directory).

By including these lines, Git helps you understand the context of the changes, making it easier to see where the changes occur within the file.

Case2: To see the difference in File Content between Working Directory and Last commit.

Note: The last commit can be referenced by **HEAD**

```
git diff HEAD file1.txt
```

Example:

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/git-practice (master)
$ git diff head file1.txt
diff --git a/file1.txt b/file1.txt
index d37b670..78ed7b9 100644
--- a/file1.txt
+++ b/file1.txt
@@ -1,4 +1,6 @@
  First line in file1.txt
  Second line in file1.txt
  Third line in file1.txt
- Fourth line in file1.txt
 \ No newline at end of file
+ Fourth line in file1.txt
+ Fifth line in file1.txt
```

```
+Sixth line in file1.txt
```

Case3: To see the difference in File Content between Staged copy and Last commit.

```
git diff --staged HEAD file1.txt
```

```
//OR
```

```
git diff --staged file1.txt
```

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/git-practice (master)
$ git diff --staged file1.txt
diff --git a/file1.txt b/file1.txt
index d37b670..e05f0a5 100644
--- a/file1.txt
+++ b/file1.txt
@@ -1,4 +1,5 @@
  First line in file1.txt
  Second line in file1.txt
  Third line in file1.txt
- Fourth line in file1.txt
\ No newline at end of file
+ Fourth line in file1.txt
+ Fifth line in file1.txt
\ No newline at end of file
```

Case4: To see the difference in File Content between specific Commit and Working Directory

```
git diff 4or7characters_of_specified_commitid filename
```

Example:

```
git log --oneline
```

```

ratan@Ratan MINGW64 ~/OneDrive/Desktop/git-practice (master)
$ git diff 0967 file1.txt
diff --git a/file1.txt b/file1.txt
index 56184bf..78ed7b9 100644
--- a/file1.txt
+++ b/file1.txt
@@ -1,2 +1,6 @@
 First line in file1.txt
-Second line in file1.txt
\ No newline at end of file
+Second line in file1.txt
+Third line in file1.txt
+Fourth line in file1.txt
+Fifth line in file1.txt
+Sixth line in file1.txt

```

Case-5: To see the difference in file content between specific commit and staging area copy

git diff --staged 4or7_charecter_of_specified_commit file_name

Example:

```

ratan@Ratan MINGW64 ~/OneDrive/Desktop/git-practice (master)
$ git diff --staged 0967 file1.txt
diff --git a/file1.txt b/file1.txt
index 56184bf..e05f0a5 100644
--- a/file1.txt
+++ b/file1.txt
@@ -1,2 +1,5 @@
 First line in file1.txt
-Second line in file1.txt
\ No newline at end of file
+Second line in file1.txt

```

```
+Third line in file1.txt
+Fourth line in file1.txt
+Fifth line in file1.txt
\ No newline at end of file
```

Case-6: To see the difference in File Content between 2 specified Commits:

```
git log --oneline
```

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/git-practice (master)
$ git diff 26a7 0967 file1.txt
diff --git a/file1.txt b/file1.txt
index d37b670..56184bf 100644
--- a/file1.txt
+++ b/file1.txt
@@ -1,4 +1,2 @@
 First line in file1.txt
-Second line in file1.txt
-Third line in file1.txt
-Fourth line in file1.txt
\ No newline at end of file
+Second line in file1.txt
\ No newline at end of file
```

Case-7: To see the difference in File Content between Last Commit and Last but one Commit(one commit before last commit)

```
git diff HEAD HEAD^ file1.txt
git diff HEAD HEAD^1 file1.txt
git diff HEAD HEAD~1 file1.txt
```

HEAD Reference to last commit

HEAD^ or HEAD^1 or HEAD~1 Reference to last but one commit

Example:

```
git diff HEAD HEAD~ file1.txt
```

Case-8: To see the differences in all Files Content between 2 specified Commits

```
git diff 26a7 0967
```

Case-9: To see the differences in Content between 2 Branches

```
$ git diff master test
```

It shows all differences between **master** branch and **test** branch

Case-10: To see the differences in Content between Local and Remote Repositories

```
$ git diff master origin/master
```

It shows all differences between master branch in local repository and master branch in remote repository.

8. git ls-files

This command in Git is used to show information about files in the index (staging area) and the working directory. It can be very useful for various tasks, such as listing files that are tracked by Git, checking which files are staged for commit, and more.

Example:

- a. To list out all the tracked file, tracked by GIT,

A tracked file may:

- Be staged
- Be committed
- Be modified
- Be unchanged

git ls-files

- b. To list out all untracked files, which are not added inside the staging area.

git ls-files --others

- c. To list out all the modified files (not yet staged for commit)

git ls-files -m

- d. List deleted files (files that are removed from the working directory but still tracked):

git ls-files -d

Undo operation:

9. git reset

git reset command is just like the reset setting in our mobile.

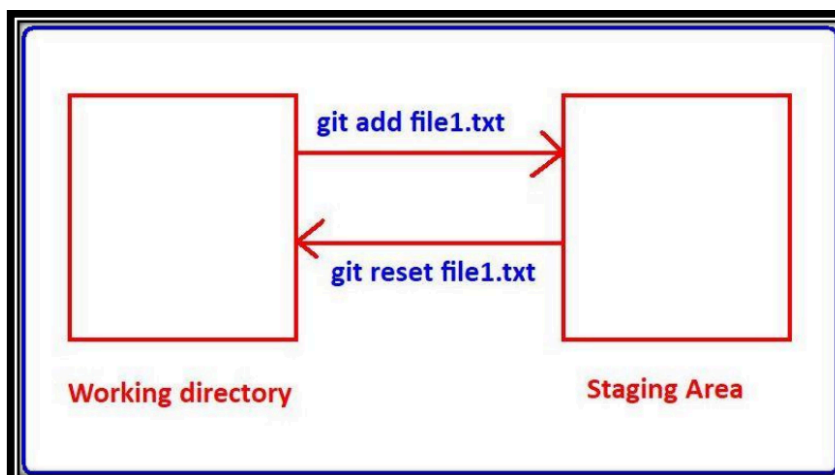
There are 2 uses of **git reset** command:

Uses1: To remove changes from staging area

Uses2: To undo commits at repository level.

Uses1: To remove changes from staging area

- If changes already added to staging area, but if we don't want to commit, then to remove such type of changes from staging area, we should go for **git reset**
- It will bring the changes from staging area back to working directory.
- It is opposite to the git add command.



- Create another folder on desktop **Git-project1** and open this folder inside the vs-code.
- Initialize this folder as a git local repository using **git init** command
- And create an index.html file with the following content:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <h1>Welcome to HDFC</h1>
</body>
</html>
```

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-project1 (master)
```

```
$ git init
```

```
Reinitialized existing Git repository in
```

```
C:/Users/ratan/OneDrive/Desktop/Git-project1/.git/
```

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-project1 (master)
```

```
$ git status
```

```
On branch master
```

```
No commits yet
```

```
Untracked files:
```

```
(use "git add <file>..." to include in what will be committed)
```

```
index.html
```

```
nothing added to commit but untracked files present (use "git add" to track)
```

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-project1 (master)
```

```
$ git add index.html
```

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-project1 (master)
```

```
$ git status
```

```
On branch master
```

```
No commits yet
```

```
Changes to be committed:
```

```
(use "git rm --cached <file>..." to unstage)
```

```
new file:   index.html
```

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-project1 (master)
```

```
$ git reset index.html
```

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-project1 (master)
```

```
$ git status
```

```
On branch master
```

```
No commits yet
```

```
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        index.html

nothing added to commit but untracked files present (use "git add" to
track)

ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-project1 (master)
$ git ls-files

ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-project1 (master)
$ git ls-files --others
index.html
```

Uses2: To undo commit at repository level

We can also use **git reset** command to undo commits at repository level

Syntax:

git reset <mode> <commitId>

It moves the HEAD to the specified commit, and all the remaining recent commits will be removed.

The **mode** will decide whether these changes are going to be removed from the staging area and working directory or not.

The allowed values for the **mode** are:

- mixed**
- soft**
- hard**

1. --mixed:

- It is the default mode.
- To discard commits in the local repository and to discard changes in the staging area.
- It won't touch the working directory.

Example:

- Commit the above created index.html inside the local repository.

```
$ git add .  
$ git commit -m "index.html file is added"
```

- Create about.html file inside the **Git-project1** folder

```
<!DOCTYPE html>  
<html lang="en">  
  
  <head>  
    <meta charset="UTF-8">  
    <meta name="viewport" content="width=device-width,  
initial-scale=1.0">  
    <title>Document</title>  
  </head>  
  
  <body>  
  
    <h1>This is a about file</h1>  
  
  </body>  
  
</html>
```

- Add this file inside the staging area and commit inside the local repository with a message.

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-project1 (master)  
$ git add about.html
```

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-project1 (master)
$ git commit -m "about.html is added"
[master bb8a83f] about.html is added
1 file changed, 16 insertions(+)
create mode 100644 about.html
```

Write git command to see the commit history; observe the output very carefully i.e. check if the latest commit is listed or not

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-project1 (master)
$ git log --oneline
bb8a83f (HEAD -> master) about.html is added
fd1cd0d index.html file is added
```

To discard the latest commit:

```
git reset --mixed fd1c
git reset fd1c

git reset --mixed HEAD~1
git reset HEAD~1
```

To discard the 2nd last commit:

```
git reset --mixed <commit-id>
git reset <commit-id>

git reset --mixed HEAD~2
git reset HEAD~2
```

The above command will move the HEAD pointer one commit back. and will leave the changes from the latest commit unstaged but present in your working directory.

So it will discard the changes from the local repository and discard the changes from the staging area also.

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-project1 (master)
$ git reset HEAD~1

ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-project1 (master)
$ git log --oneline
fd1cd0d (HEAD -> master) index.html file is added

ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-project1 (master)
$ git status
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    about.html

nothing added to commit but untracked files present (use "git add" to track)
```

After undo commit-2

- The changes will be there inside the working directory, so we can modify those changes.
- We can do the modification inside the about.html
- We need to add this file inside the staging area
- We need to commit with the appropriate message

Add following line inside the **about.html** file

```
<h1>New Line is added</h1>
```

Adding this file inside the staging area and commit the changes with message:

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-project1 (master)
$ git add about.html

ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-project1 (master)
$ git commit -m "about.html file is committed after modification";
[master c17adb3] about.html file is committed after modification
1 file changed, 16 insertions(+)
create mode 100644 about.html

ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-project1 (master)
$ git status
On branch master
nothing to commit, working tree clean

ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-project1 (master)
$ git log --oneline
c17adb3 (HEAD -> master) about.html file is committed after
modification
fd1cd0d index.html file is added
```

2. --soft:

- It is exactly same as **--mixed** option, but changes are available in working directory as well as in the staging area.
- It won't touch the staging area and working directory.
- As changes are already present in the staging area, we just have to use commit once again to get it back inside the local repository.

Example:

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-project1 (master)
$ git reset --soft HEAD~1

ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-project1 (master)
$ git log --oneline
```

```
fd1cd0d (HEAD -> master) index.html file is added

ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-project1 (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   about.html

ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-project1 (master)
$ git ls-files
about.html
index.html
```

Use Cases:

1. If some files are missing in the last commit, then add those files and commit again.
2. If we forgot to add some important points inside the commit message.

3. --hard:

- It is exactly same as --mixed except that Changes will be removed from everywhere (local repository, staging area, working directory)
- It is a more dangerous command and it is a destructive command.
- It is impossible to revert back our code and hence while using hard reset we have to take special care.

Example:

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-project1 (master)
$ git commit -m "about.html is committed once again"
[master 3842570] about.html is committed once again
1 file changed, 18 insertions(+)
create mode 100644 about.html
```

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-project1 (master)
$ git log --oneline
3842570 (HEAD -> master) about.html is committed once again
fd1cd0d index.html file is added
```

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-project1 (master)
$ git reset --hard HEAD~1
HEAD is now at fd1cd0d index.html file is added
```

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-project1 (master)
$ git log --oneline
fd1cd0d (HEAD -> master) index.html file is added
```

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-project1 (master)
$ git status
On branch master
nothing to commit, working tree clean
```

git reflog:

- Git keeps track of every movement of HEAD.
- Even if commits disappear from history, Git remembers them **temporarily**.

Example:

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/git-practice1 (master)
$ git reflog
fac50d7 (HEAD -> master) HEAD@{0}: reset: moving to HEAD~1
4ff2dbe HEAD@{1}: commit: about.html is added and committed
fac50d7 (HEAD -> master) HEAD@{2}: reset: moving to HEAD~1
9323c8a HEAD@{3}: commit: about.html file is committed after modification
fac50d7 (HEAD -> master) HEAD@{4}: reset: moving to HEAD
fac50d7 (HEAD -> master) HEAD@{5}: reset: moving to fac5
4036c22 HEAD@{6}: reset: moving to 4036
4036c22 HEAD@{7}: commit: about.html is added
fac50d7 (HEAD -> master) HEAD@{8}: commit (initial): index.html file is added
```

Restore:

git reset --hard 4ff2dbe

Using git reflog,

Git **often** keeps a history of previous HEAD positions, allowing lost commits to be recovered.

Typically:

Reflog entries remains :90 days

for reachable commits.

Summery : --mixed, --soft, --hard

a. --mixed:

- changes will be discarded in the local repo and staging area.
- It won't touch working directly.
- The working tree won't be clean.
- But we can revert with
 - git add .
 - git commit

b. --soft:

- changes will be discarded only in the local repository.
- It won't touch the staging area and working directory.
- The working tree won't be clean.
- But we can revert with
 - git commit

c. --hard:

- changes will be discarded everywhere.

- The working tree will be clean.
- No way to revert (We can revert it temporarily using **reflog**).

Note:

- If the commits are confirmed to the local repository and to discard those commits we can use the reset command.
- If the commits have already been pushed to a remote repository and are being used by other developers, it is generally not recommended to use git reset. Instead, use **git revert**.

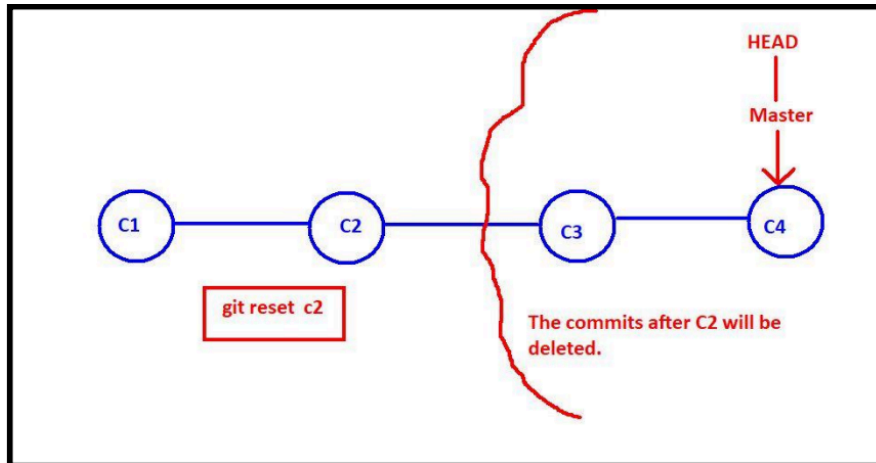
Reason:

- **git reset** rewrites commit history, which can create problems for other developers who have already pulled those commits.
- **git revert** is safer because it creates a new commit that undoes the changes without modifying existing history.

10. git revert

Understanding the need for the **git revert** command:

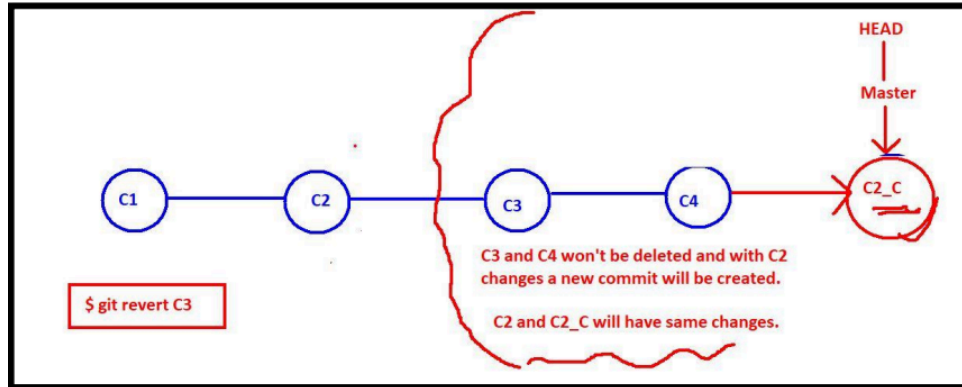
- Suppose we have 4 commits are there in a repository (c1, c2, c3 ,c4)
- And we reset to commit 2
 - **git reset c2**
- After c2 all the next commits will be deleted.



- Suppose if this code already pushed to the **remote** repository and already several people pulled this code and started working on that code, **git reset** command may create a problem.
- **git reset** command deletes commit history and hence it is destructive command and not recommended to use on public repositories.
- To overcome this problem we have to go for the **git revert** command.
- **git revert** command won't delete commit history. It reverts the required commit by creating a **new commit**. i.e it will undo a particular commit without deleting commit history.

```
$ git revert C3
```

git will revert the changes of C3, meaning with the state of C2, a new commit will be created.



In **git revert**, commit history won't be deleted and hence there is no effect on peer developers. It is a safe operation.

Example:

Till now inside our local repository, only one commit is there, we have reset hard to discard the 2nd commit.

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-project1 (master)
$ git log --oneline
fd1cd0d (HEAD -> master) index.html file is added

ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-project1 (master)
$ git status
On branch master
nothing to commit, working tree clean
```

Now create **sitemap.html** file inside the vs-code:

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>

<body>
```

```
<a href='#'>Home</a>
<a href='#'>About</a>
<a href='#'>Contact</a>

</body>

</html>
```

Add the above file inside the staging area and commit this file with proper message.

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-project1 (master)
$ git add sitemap.html

ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-project1 (master)
$ git commit -m "sitemap.html is created and committed"
[master 6c6ccb3] sitemap.html is created and committed
1 file changed, 18 insertions(+)
create mode 100644 sitemap.html

ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-project1 (master)
$ git log --oneline
6c6ccb3 (HEAD -> master) sitemap.html is created and committed
fd1cd0d index.html file is added
```

Lets try to revert the last commit:

Syntax: `git revert <commit-id>`

Here the commit id is that id which we need to revert.

```
$ git revert 6c6c
```

This will ask you to write a commit message; after writing the commit message (press `i` for insert mode in the editor)(You can keep the default commit message) press `:wq` to come to the prompt again.

Write a git command to see the commit history; observe the output very carefully i.e. check if the latest commit is listed or not.

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-project1 (master)
$ git log --oneline
aa9f3b5 (HEAD -> master) Revert "sitemap.html is created and committed"
6c6ccb3 sitemap.html is created and committed
fd1cd0d index.html file is added

ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-project1 (master)
$ git status
On branch master
nothing to commit, working tree clean

ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-project1 (master)
$ dir
index.html
```

File **sitemap.html** is not in the working directory.

Note: for **git revert** command --soft, --mixed and --hard option is not applicable.

Moving inside any specific commit: getting the state of a particular commit from the local repository to inside the working directory:

11. git checkout:

- Using this command we can move inside a specific commit.
- Using this command we can switch from one branch to another also.

Moving inside a specific commit:

Syntax:

```
Git checkout <commit-hash>
```

With this we can check out a specific commit, which puts our repository in a "**detached HEAD**" state. This means you are no longer on any branch, but are viewing the repository at the state of that specific commit.

Now your HEAD will point to the **specific commit** instead of pointing to **any branch**. This is called "**detached HEAD**".

Example:

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-project1 (master)
$ git log --oneline
aa9f3b5 (HEAD -> master) Revert "sitemap.html is created and committed"
6c6ccb3 sitemap.html is created and committed
fd1cd0d index.html file is added
```

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-project1 (master)
$ git checkout 6c6c
Note: switching to '6c6c'.
```

You are in 'detached HEAD' state. You can look around, make experimental changes and commit them, and you can discard any commits you make in this state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may do so (now or later) by using -c with the switch command. Example:

```
git switch -c <new-branch-name>
```

Or undo this operation with:

```
git switch -
```

Turn off this advice by setting config variable advice.detachedHead to false

HEAD is now at 6c6ccb3 sitemap.html is created and committed

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-project1 ((6c6ccb3...))
$ git status
HEAD detached at 6c6ccb3
```

```
nothing to commit, working tree clean
```

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-project1 ((6c6ccb3...))  
$ git log --oneline  
6c6ccb3 (HEAD) sitemap.html is created and committed  
fd1cd0d index.html file is added
```

While in the detached HEAD state, you can explore the repository, inspect files, and even make changes. However, if you make changes and want to save them, you need to create a new branch or commit those changes in a way that ensures they are not lost.

Create a New Branch (if needed):

If you decide you want to make changes and keep them, create a new branch from this commit.

Creating a new branch called feature-branch and switching inside this branch.

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-project1 ((6c6ccb3...))  
$ git branch feature-branch  
  
ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-project1 ((6c6ccb3...))  
$ git switch feature-branch  
Switched to branch 'feature-branch'  
  
ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-project1 (feature-branch)  
$ git branch  
* feature-branch  
  master
```

Please observe the above output carefully.

Here * symbol represent, it is a currently active branch.

git branch <branch-name>: to create a new branch.

git checkout <branch-name>: to move inside a specific branch.

OR

git switch <branch-name>: to move inside a specific branch.

We can use the following command as a shortcut for creating a new branch and move inside that branch.

git checkout -b <branch-name>

OR

git switch -c <branch-name>

Returning to the previous state:

To return to the previous state (typically your master branch or another branch), simply switch that branch again.

```
git switch master
```

GitIgnore:

To ignore some files and folders to be tracked by the Git version control system, we can use this GitIgnore concept.

In this concept we need to create one file with the name **.gitignore** and specify all the files and folders which we do not want to be tracked by the git, inside this **.gitignore** file.

Purpose and benefits:

1. **Avoid making repositories noisy:** Prevents temporary or build files from cluttering the repository.
2. **Security:** Keeps sensitive files like configuration files containing secrets or

credentials out of the repository.

3. **Efficiency:** Reduces the size of the repository by ignoring large files that are not necessary for version control.

Example: create a **.gitignore** file inside the working directory.

```
# Ignore a specific file
secret.txt

# Ignore the entire folder
/logs

# Ignore all .log files (using a file pattern)
*.log

# Ignore all .log files in the logs folder (ignoring files in a specific folder)
logs/*.log

# Ignore all files in any folder named tmp
tmp/*

# Node.js dependencies
node_modules/
```

Commit this file with a proper message, and try to create a **secret.txt** file to verify whether this file is being tracked or not.

Example of SpringBoot related .gitignore file:

```
target/

.idea/
```

```
.vscode/
```

```
*.log
```

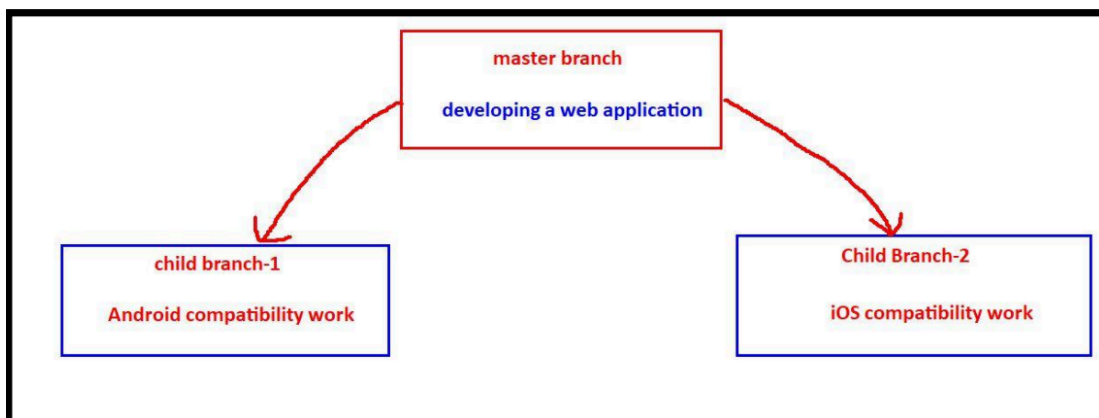
```
.env
```

Git Branching and Merging:

- Branching is one of the most important concepts in version control systems.
- Till now whatever files created and whatever commits we did, all these happened in the master/main branch.
- master/main branch is the default branch in git.
- Generally the main source code will be placed in the master/main branch.

Need of creating a New Branch:

Assume we required to work on new requirements independently, then instead of working in the master branch, we can create a separate branch and we can work in that branch, related to that new requirement without affecting the main branch.

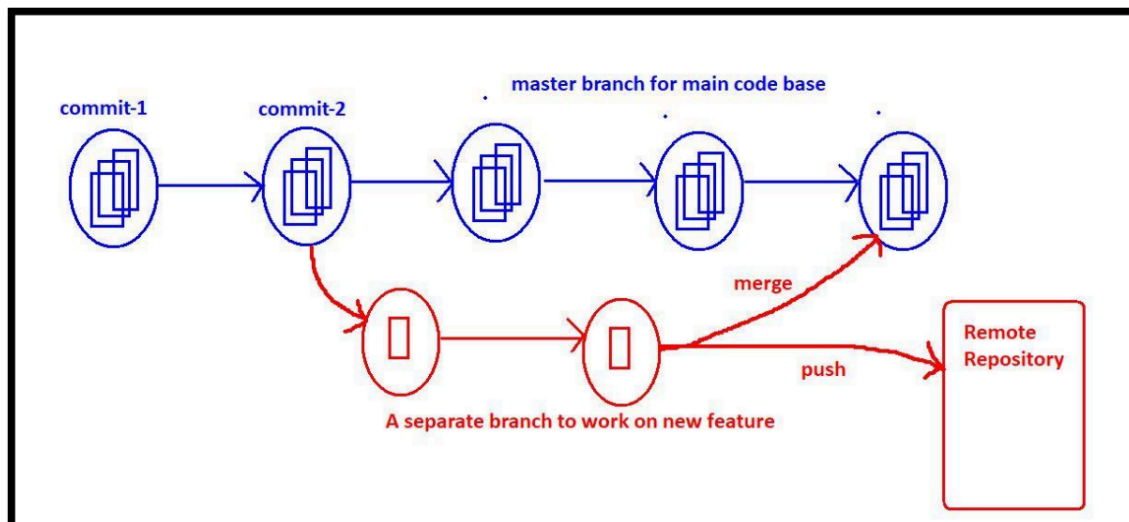


master/main branch => develop web application

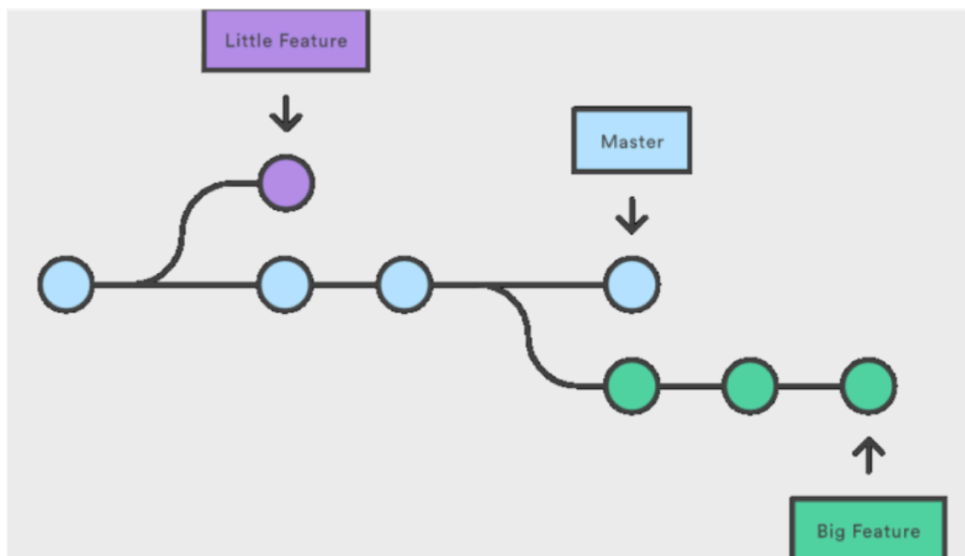
branch-1 => develop Android compatibility work

branch-2 => develop iOS compatibility work

A Branch is nothing but an independent flow of development and by using branching concept, we can create multiple work flows.



Based on our requirement, we can create any number of branches



Conclusions:

1. Once we create a branch all files and commits will be **inherited** from parent branch to child branch. Branching is a logical way of duplicating files and commits. In the child branch we can create new files and we can perform new commits based on our requirements.
2. All branches are isolated from each other. The changes performed in master branch are not visible to the new branch and the changes performed in the new branch are not visible to the master branch.
3. Once the work is completed in new branch then we can merge that new branch to the main branch or we can push that branch directly to the remote repository.

Various commands used in branching:

1. To View Branches:

git branch : To know all available branches in our local repository

Note: There is another way to check on which branch currently we are working, for this we have to use the **git status** command.

Example:

```
$ git status
On branch master
nothing to commit, working tree clean
```

2. To create a new branch:
git branch <branch-name>
3. To Switch from one Branch to another Branch:
git switch <branch-name>

OR

git checkout <branch-name>

4. Short-cut Way to Create a New Branch and switch to that branch:

git checkout -b <branch-name>

OR

git switch -c <branch-name>

Note: In GIT branching, logical duplication of files will happen. For every branch, a new directory won't be created.

But in other version control systems like SVN, if we want to create a branch, first we have to create a new directory and we have to copy all files manually to that directory which is a very difficult and time consuming job.

Advantages of branching:

1. We can enable Parallel development.
2. We can work on multiple flows in an isolated way.
3. We can organize source code in a clean way.
4. Implementing new features will become easy
5. Bug fixing will become easy.
6. Testing new ideas or new technologies will become easy

Demo Example for branching:

Create a separate folder inside the desktop **git-branch-demo** and open this folder inside the vs-code. And initialize this folder as a local repository.

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/git-branch-demo (master)
$ git init
```

Create an **index.html** file and commit this file inside the local repository.

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>

<body>

  <h1>Welcome to HDFC</h1>

</body>

</html>
```

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/git-branch-demo (master)
$ git add index.html
```

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/git-branch-demo (master)
$ git commit -m "index.html file is created and committed"
[master (root-commit) b0f66ef] index.html file is created and
committed
1 file changed, 13 insertions(+)
create mode 100644 index.html
```

Note: inside the local repository one default branch is created with the name **master**

To list out all the available branches

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/git-branch-demo (master)
$ git branch
* master
```

- The * On master means all this is current working branch i.e. all work that we have done so far and doing now are part of master branch.
- The term "master" carries historical baggage and has connotations that may be offensive or insensitive to some people, particularly due to its association with slavery and oppression.
- The renaming of the default branch from "**master**" to "**main**" in Git is part of an effort to make the language used in software development more inclusive and mindful of historical context.
- On the GitHub (Web repository) the branch is **main** not **master** so we have to use the same name for both local repository and remote repository for better synchronization.

To rename the branch **master** to **main**

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/git-branch-demo (master)
$ git branch -m master main

ratan@Ratan MINGW64 ~/OneDrive/Desktop/git-branch-demo (main)
$ git branch
* main
```

Say we have a new developer who wants to create a new web page; We want him to work in a separate branch so that he should not interfere with code already written in the **main** branch. Let us create a new branch for him.

Write git command to create a new branch named 'developer_01'

Syntax:

```
git branch <branch-name>
```

Example:

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/git-branch-demo (main)
$ git branch developer1

ratan@Ratan MINGW64 ~/OneDrive/Desktop/git-branch-demo (main)
$ git branch
  developer1
* main
```

Write git command to switch to **developer1** branch [Now all work will be in the developer1 branch; main branch will be unaffected now]

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/git-branch-demo (main)
$ git switch developer1
Switched to branch 'developer1'

ratan@Ratan MINGW64 ~/OneDrive/Desktop/git-branch-demo (developer1)
$ git branch
* developer1
  main
```

Here all the commits and codes from the master branch will be copied inside the developer1 branch.

Now create a new html file called **gallery.html** and commit it.

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
```

```
<body>
  <h1>This is a gallery.html file</h1>

</body>

</html>
```

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/git-branch-demo (developer1)
$ git add gallery.html
```

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/git-branch-demo (developer1)
$ git commit -m "gallery.html file is created and committed"
[developer1 387449d] gallery.html file is created and committed
1 file changed, 12 insertions(+)
create mode 100644 gallery.html
```

Write **git log** command to see a commit history; observe the output very carefully

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/git-branch-demo (developer1)
$ git log
commit 387449d37724a27645758b7008e9bc786ea3ed70 (HEAD -> developer1)
Author: Ratan <ratankapr22@gmail.com>
Date: Sun May 26 18:05:45 2024 +0530
```

```
    gallery.html file is created and committed
```

```
commit b0f66ef890105b75f3f81304f38de3fc59ba9007 (main)
Author: Ratan <ratankapr22@gmail.com>
Date: Sun May 26 17:50:53 2024 +0530
```

```
    index.html file is created and committed
```

The above commit history has details from the main branch also.

Now suppose from this developer1 branch, another junior-developer wants to create a new html file in such a way that the flow/work of the developer1 should not be affected. So junior-developers will create a separate branch from this developer1 branch.

git command to create and switch to a new branch named **junior-developer1** using a single command:

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/git-branch-demo (developer1)
$ git switch -c junior-developer1
Switched to a new branch 'junior-developer1'

ratan@Ratan MINGW64 ~/OneDrive/Desktop/git-branch-demo (junior-developer1)
$ git branch
  developer1
* junior-developer1
  main
```

Now junior-developer1 will create a new html file called **contact.html** inside his branch:

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>

<body>
  <h1>This is a contact1.html</h1>
</body>

</html>
```

Now lets commit this contact.html by adding it inside the staging area:

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/git-branch-demo (junior-developer1)
$ git add contact.html

ratan@Ratan MINGW64 ~/OneDrive/Desktop/git-branch-demo (junior-developer1)
$ git commit -m "contact.html file is created and committed"
[junior-developer1 3369926] contact.html file is created and committed
1 file changed, 15 insertions(+)
create mode 100644 contact.html
```

Write git command to see commit history

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/git-branch-demo (junior-developer1)
$ git log
commit 3369926475bd9e7dbba50ec43683ad778433763e (HEAD -> junior-developer1)
Author: Ratan <ratankapr22@gmail.com>
Date: Sun May 26 18:18:11 2024 +0530

    contact.html file is created and committed

commit 387449d37724a27645758b7008e9bc786ea3ed70 (developer1)
Author: Ratan <ratankapr22@gmail.com>
Date: Sun May 26 18:05:45 2024 +0530

    gallery.html file is created and committed

commit b0f66ef890105b75f3f81304f38de3fc59ba9007 (main)
Author: Ratan <ratankapr22@gmail.com>
Date: Sun May 26 17:50:53 2024 +0530

    index.html file is created and committed
```

- This has commit history from all parent branches of itself i.e. it has commit history from **developer1** branch and the **main** branch.

- The commit history for the child branch will not be visible to its parent branch.

Example:

Lets move to the **developer1** branch:

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/git-branch-demo (junior-developer1)
$ git switch developer1
M      gallery.html
M      index.html
Switched to branch 'developer1'
```

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/git-branch-demo (developer1)
$ git log
commit 387449d37724a27645758b7008e9bc786ea3ed70 (HEAD -> developer1)
Author: Ratan <ratankapr22@gmail.com>
Date:   Sun May 26 18:05:45 2024 +0530
```

gallery.html file is created and committed

```
commit b0f66ef890105b75f3f81304f38de3fc59ba9007 (main)
Author: Ratan <ratankapr22@gmail.com>
Date:   Sun May 26 17:50:53 2024 +0530
```

index.html file is created and committed

Notice, here **junior-developer1** branch related commit is not visible and we can notice the file created by the **junior-developer1** branch is also removed from the vs-code.

Now if **developer1** try to merge the code written by the junior-developer then he can merge it by using following command:

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/git-branch-demo (developer1)
$ git merge junior-developer1
Updating 387449d..3369926
Fast-forward
```

```
contact.html | 15 ++++++++
1 file changed, 15 insertions(+)
create mode 100644 contact.html
```

To merge any child branch we need to first move inside the parent branch then only we can merge its child branch.

Now all the commits for the **junior-developer1** are visible to the **developer1** branch.

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/git-branch-demo (developer1)
$ git log
commit 3369926475bd9e7dbba50ec43683ad778433763e (HEAD -> developer1, junior-developer1)
Author: Ratan <ratankapr22@gmail.com>
Date: Sun May 26 18:18:11 2024 +0530

    contact.html file is created and committed

commit 387449d37724a27645758b7008e9bc786ea3ed70
Author: Ratan <ratankapr22@gmail.com>
Date: Sun May 26 18:05:45 2024 +0530

    gallery.html file is created and committed

commit b0f66ef890105b75f3f81304f38de3fc59ba9007 (main)
Author: Ratan <ratankapr22@gmail.com>
Date: Sun May 26 17:50:53 2024 +0530

    index.html file is created and committed
```

Now lets move to the **main** branch and merge the **developer1** branch.

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/git-branch-demo (developer1)
$ git switch main
Switched to branch 'main'
```

```

ratan@Ratan MINGW64 ~/OneDrive/Desktop/git-branch-demo (main)
$ git merge developer1
Updating b0f66ef..bbb684c
Fast-forward
 contact.html | 15 +++++
 gallery.html | 15 +++++
 index.html   |  5 +++-
3 files changed, 34 insertions(+), 1 deletion(-)
create mode 100644 contact.html
create mode 100644 gallery.html

ratan@Ratan MINGW64 ~/OneDrive/Desktop/git-branch-demo (main)
$ git status
On branch main
nothing to commit, working tree clean

```

Here all the commits related to the **developer1** and **junior-developer1** branch and all the codes for these branches are available to the **main** branch.

Now we can delete the junior-developer1 and developer1 branch from the main branch

```

ratan@Ratan MINGW64 ~/OneDrive/Desktop/git-branch-demo (junior-developer1)
$ git switch main
Switched to branch 'main'

ratan@Ratan MINGW64 ~/OneDrive/Desktop/git-branch-demo (main)
$ git branch -d junior-developer1
Deleted branch junior-developer1 (was b7c9aa7).

ratan@Ratan MINGW64 ~/OneDrive/Desktop/git-branch-demo (main)
$ git branch -d developer1
Deleted branch developer1 (was cda084f).

```

Types of Merging in Git:

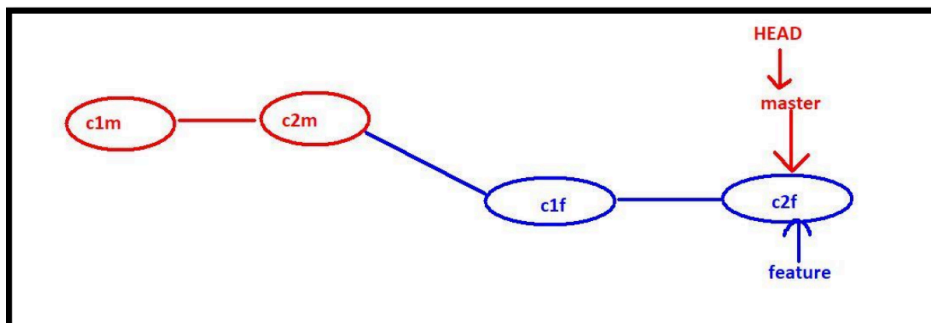
- There are 2 types of merging in Git.

1. Fast-forward merge
2. Three-Way merge

What is Fast-forward Merge?

After creating child branch, if we are not doing any new commits in the parent branch, then git will perform fast-forward merge. i.e updations(new commits) happened only in child branch but not in parent branch.

In the fast-forward merge, git simply moves parent branch and points to the last commit of the child branch.



If the same file is modified by both parent and child branches then conflicts will be raised. In fast-forward merge there is no chance of any conflicts, because updations happened only in the child branch and we didn't touch the parent branch.

Note: After creating child branch if parent branch also contains some new commits, then

Fast-forward merge won't happen and **Three-way merge** will happen.

Three-way merge:

If changes present in both parent and child branches and if we are trying to perform merge operation, then git will do **three-way merge**.

Demo Example to demonstrate Three-way merge:

Create a folder **Git-branch-demo2** on the desktop and open this inside the vs-code.

Initialize this folder as a git local repository:

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-branch-demo2
$ git init
Initialized empty Git repository in
C:/Users/ratan/OneDrive/Desktop/Git-branch-demo2/.git/
```

Rename the **master** branch to the **main** branch.

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-branch-demo2 (master)
$ git branch -m master main
```

Create 2 empty files inside this folder called: **a.txt** and **b.txt**

Commit a.txt with message: **c1m**

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-branch-demo2 (master)
$ git add a.txt

ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-branch-demo2 (master)
$ git commit -m "c1m"
[master (root-commit) b7dcd8c] c1m
1 file changed, 0 insertions(+), 0 deletions(-)
```

```
create mode 100644 a.txt
```

Commit b.txt with message: **c2m**

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-branch-demo2 (master)
$ git add b.txt

ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-branch-demo2 (master)
$ git commit -m "c2m"
[master c92bad8] c2m
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 b.txt
```

Check all the commits performed in **main** branch:

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-branch-demo2 (main)
$ git log --oneline
eee6be3 (HEAD -> main) c2m
086e8ad c1m
```

Create a new branch from this **main** branch called **feature** and switch inside the **feature** branch.

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-branch-demo2 (main)
$ git switch -c feature
Switched to a new branch 'feature'

ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-branch-demo2 (feature)
$ git branch
* feature
  main
```

Create 2 empty files **x.txt** and **z.txt** inside the **feature** branch.

Now commit x.txt with message: **c1f**

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-branch-demo2 (feature)
```

```
$ git add x.txt
```

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-branch-demo2 (feature)
```

```
$ git commit -m "c1f"
```

```
[feature 46fc37c] c1f
```

```
1 file changed, 0 insertions(+), 0 deletions(-)
```

```
create mode 100644 x.txt
```

Commit z.txt with message: **c2f**

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-branch-demo2 (feature)
```

```
$ git add z.txt
```

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-branch-demo2 (feature)
```

```
$ git commit -m "c2f"
```

```
[feature aff4098] c2f
```

```
1 file changed, 0 insertions(+), 0 deletions(-)
```

```
create mode 100644 z.txt
```

Log all the commits of the feature branch, here we will see all the commits of the main branch are inherited.

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-branch-demo2 (feature)
```

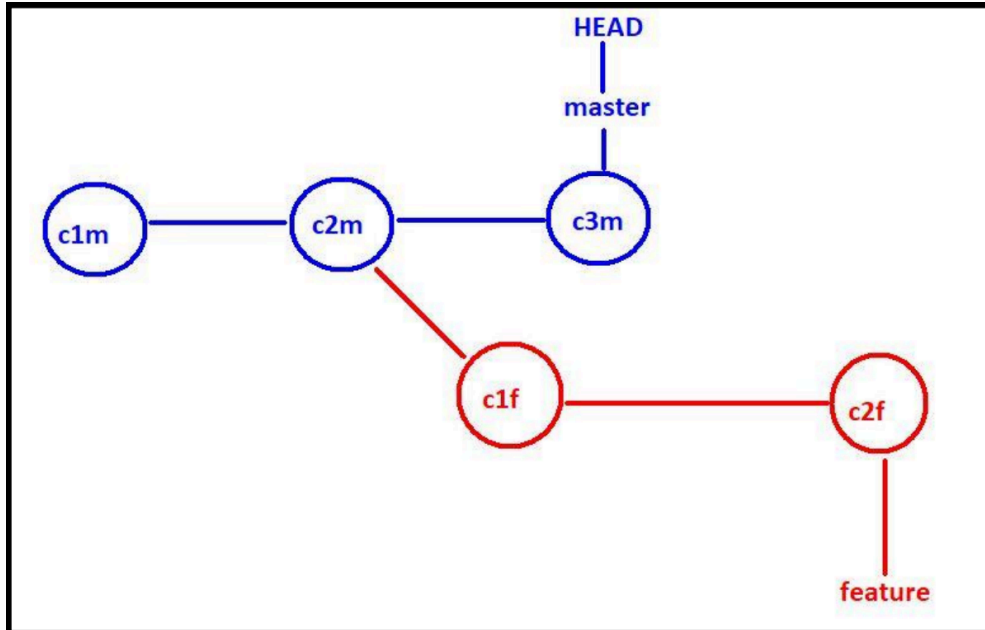
```
$ git log --oneline
```

```
aff4098 (HEAD -> feature) c2f
```

```
46fc37c c1f
```

```
eee6be3 (main) c2m
```

```
086e8ad c1m
```



Now switch to the **main** branch:

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-branch-demo2 (feature)
$ git switch main
Switched to branch 'main'
```

Here we can see the files created by the **feature** branch are removed from the vs-code.

Create an empty file **c.txt** inside the **main** branch.

And commit this file in **main** branch with message: **c3m**

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-branch-demo2 (main)
$ git add c.txt

ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-branch-demo2 (main)
$ git commit -m "c3m"
[main 182d969] c3m
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 c.txt
```

In this case new commits are not available in child branches. And if we are trying to perform a merge operation, git will do a three-way **merge**.

While merging it may ask to insert any message (why we are merging the child branch, but we can ignore it with default message and quit the editor by typing **:wq**)

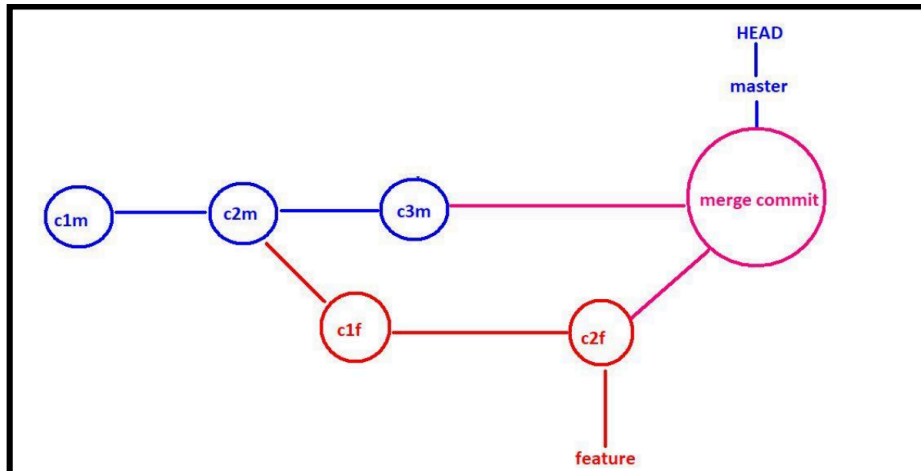
```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-branch-demo2 (main)
$ git merge feature
Merge made by the 'ort' strategy.
 x.txt | 0
 z.txt | 0
2 files changed, 0 insertions(+), 0 deletions(-)
create mode 100644 x.txt
create mode 100644 z.txt
```

Three-way merge creates a new commit which is also known as merge commit. The parent branch will point to the newly created merge commit.

Here ORT stands for "**Ostensibly Recursive's Twin**," this is the new strategy introduced in Git 2.33 version, previously it was only **Recursive** strategy.

The **Ort strategy** was designed to be more efficient and provide better handling of complex merge scenarios compared to the older **recursive strategy**.

We can see the files created by feature branches i.e. x.txt and z.txt is now visible to the main branch and inside the vs-code.



```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-branch-demo2 (main)
$ git log --oneline
ece5d49 (HEAD -> main) Merge branch 'feature'
182d969 c3m
aff4098 (feature) c2f
46fc37c c1f
eee6be3 c2m
086e8ad c1m
```

Here we can see all the commits of feature branch is included inside the main branch and one new commit also created for this merge with default message: **Merge branch 'feature'**

Difference between **fast-forward** and **Three-way merge**:

Fast-forward	Three-way Merge
1. After creating child branch, if updates are available only in the child branch but not in the parent branch, then GIT will perform Fast-Forward Merge.	1. After creating child branch, if updates are available in both Parent and child branches, then GIT will perform Three-way Merge.
2. It does not require any additional commit	2. It requires a new commit which is also known as Merge commit.
3. There is no chance of conflicts because new commits are available only in child branch but not in parent branch.	3. There may be a chance of conflicts because new commits are available in both parent and child branches.
4. Fast-forward merge is fully handled by GIT.	4. If there is a conflict, we may required to handle manually.

Merge Conflicts and Resolution Process:

- In the case of 3-way merge, if the same file is updated by both Parent and child branches then there may be a chance of merge conflict.
- If there is a conflict then GIT stops the merge process and provides a conflict message.
We have to resolve the conflict manually by editing the file.
- Git will markup both branches content in the file to resolve the conflict very easily.
- Once we completed editing of the file with required final content, then we have to add to

the staging area followed by a commit. With that, the merging process will be completed.

Demo Example:

Create a folder **Git-branch-demo3** on the desktop and open it inside the vs-code.

Initialize this folder as git local repository:

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-branch-demo3
```

```
$ git init
Initialized empty Git repository in
C:/Users/ratan/OneDrive/Desktop/Git-branch-demo3/.git/
```

Rename the master branch to the main branch:

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-branch-demo3 (master)
$ git branch -m master main
```

Create a file called **a.txt** and add one line inside this file: **First Line Added**

Commit this file inside the main branch with the message: **c1m**

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-branch-demo3 (main)
$ git add a.txt

ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-branch-demo3 (main)
$ git commit -m "c1m"
[main (root-commit) 2615186] c1m
 1 file changed, 1 insertion(+)
 create mode 100644 a.txt
```

Edit the file a.txt and add another line: **Second Line Added**

Commit this change inside the main branch with the message: **c2m**

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-branch-demo3 (main)
$ git add a.txt

ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-branch-demo3 (main)
$ git commit -m "c2m"
[main 9f4569a] c2m
 1 file changed, 1 insertion(+)
```

Create a new branch called **feature** and switch inside this branch.

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-branch-demo3 (main)
$ git checkout -b feature
Switched to a new branch 'feature'
```

Edit the a.txt with new line: **New Data Added By Feature Branch**

Commit this changes inside the feature branch with message: **c1f**

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-branch-demo3 (feature)
$ git add a.txt

ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-branch-demo3 (feature)
$ git commit -m "c1f"
[feature d9af5ea] c1f
1 file changed, 2 insertions(+)
```

Note: this commit will not be visible to the **main** branch.

Now switch to the **main** branch.

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-branch-demo3 (feature)
$ git checkout main
Switched to branch 'main'
```

Here we can see the 3rd line from the **a.txt** is removed. Which was added by the **feature** branch.

Now edit the a.txt and add the 3rd line inside **main** branch: **New Data Added By Main Branch**

Commit this change inside the **main** branch with message: **c3m**

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-branch-demo3 (main)
```

```
$ git add a.txt
```

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-branch-demo3 (main)
```

```
$ git commit -m "c3m"
```

```
[main c49abf4] c3m
```

```
1 file changed, 1 insertion(+)
```

This commit will not be visible to the **Feature** branch.

We can see the differences between both branches: using the **git diff** command.

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-branch-demo3 (main)
```

```
$ git diff main feature
```

```
diff --git a/a.txt b/a.txt
```

```
index adefdaf..577baa2 100644
```

```
--- a/a.txt
```

```
+++ b/a.txt
```

```
@@ -1,3 +1,4 @@
```

```
First Line Added
```

```
Second Line Added
```

```
-New Data Added By Main Branch
```

```
+New Data Added By Feature Branch
```

```
+
```

Now lets try to merge the **feature** branch from the **main** branch.

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-branch-demo3 (main)
```

```
$ git merge feature
```

```
Auto-merging a.txt
```

```
CONFLICT (content): Merge conflict in a.txt
```

```
Automatic merge failed; fix conflicts and then commit the result.
```

Here the a.txt file will look like as follows:

```

First Line Added
Second Line Added
Accept Current Change | Accept Incoming Change | Accept Both Changes | Compare Changes
<<<<<< HEAD (Current Change)
New Data Added By Master Branch
=====
New Data Added By Feature Branch
>>>>>> feature (Incoming Change)

```

[Resolve in Merge Editor](#)

Lets see the status of our main branch:

```

ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-branch-demo3 (main|MERGING)
$ git status
On branch main
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add <file>..." to mark resolution)
    both modified:   a.txt

no changes added to commit (use "git add" and/or "git commit -a")

```

We have to edit a.txt to decide the final content.

We can do this by choosing the various options link showing in vs-code a.txt file (refer above image)

Accept current change | Accept Incoming change | Accept both changes

And then commit it by adding this file inside the staging area with a merging related message.

```

ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-branch-demo3 (main|MERGING)
$ git add a.txt

ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-branch-demo3 (main|MERGING)
$ git commit -m "Resolved merge conflict"
[main 1c10d8e] Resolved merge conflict

```

```
ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-branch-demo3 (main)
$ git status
On branch main
nothing to commit, working tree clean

ratan@Ratan MINGW64 ~/OneDrive/Desktop/Git-branch-demo3 (main)
$ git log --oneline
1c10d8e (HEAD -> main) Resolved merge conflict
c49abf4 c3m
d9af5ea (feature) c1f
9f4569a c2m
2615186 c1m
```

How to Delete a Branch ?:

- Once we complete our work we can delete the branch.
- Deletion of the branch is optional.
- The main objective of deleting branches is to keep our repository clean.
- We can delete a branch by using the git branch command with the -d option.

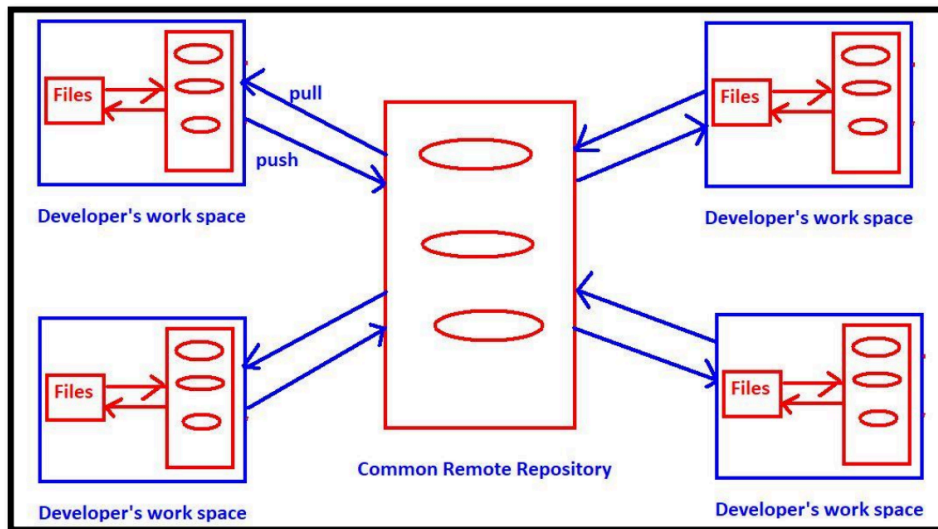
Syntax: **\$ git branch -d <branch_name>**

Remote Repository: GitHub

Need of Remote Repositories:

- Upto this we created multiple files and we did several changes. We used multiple git commands. All these operations happen in the local repository.
- If we required to share our code to the peer developers/team members then the remote repository concept will come in the picture.

- As GIT is a distributed version control system, every developer has his own local repository.
Every developer can share his code to the peer developers/team members.



Note: It is not centralized version control system

- The most common service providers of remote repositories
Github, GitLab, Bitbucket etc
- Some big companies may use their own common remote repository instead of using 3rd party repositories.

Working with GitHub:

- Github is an online cloud based GIT server where remote repositories are maintained.
- We can store our repositories in github so that our code is available for the remaining team members.
- Various Git commands to work with remote repository

git remote
git push
git clone
git pull
git fetch

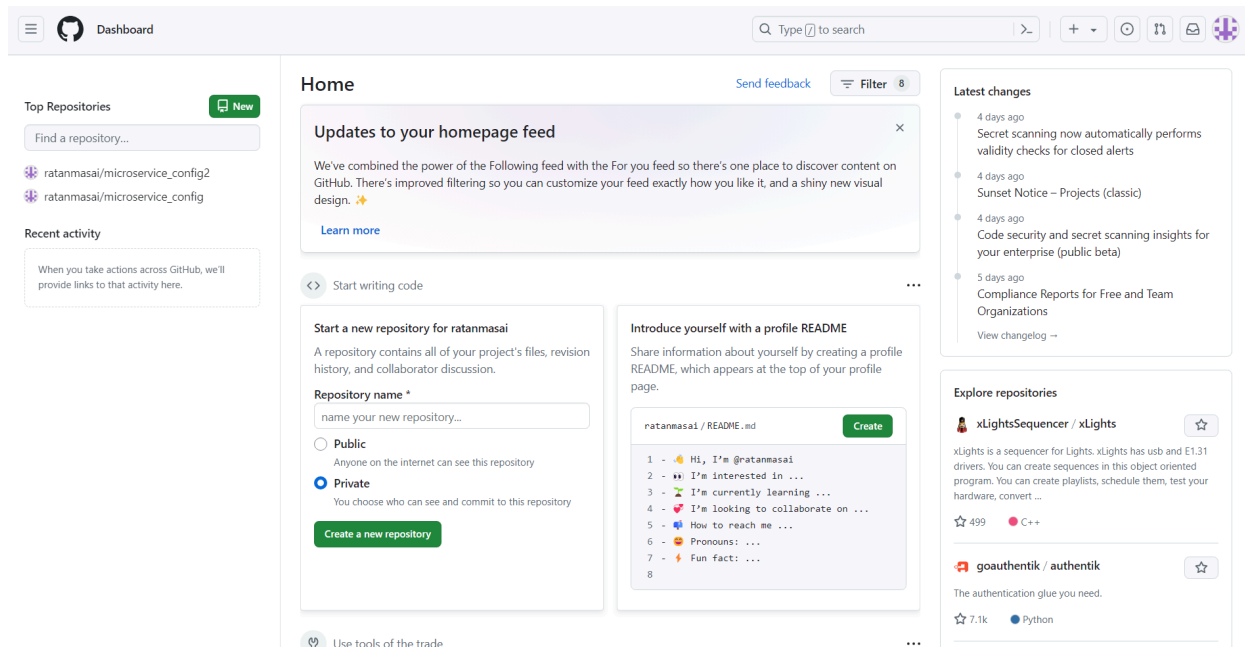
- **GitHub url:**

<https://github.com/>

We have to sign up with our mail id. We should use valid mail id only because we have to verify that mail id to create account successfully. github requires email verification.

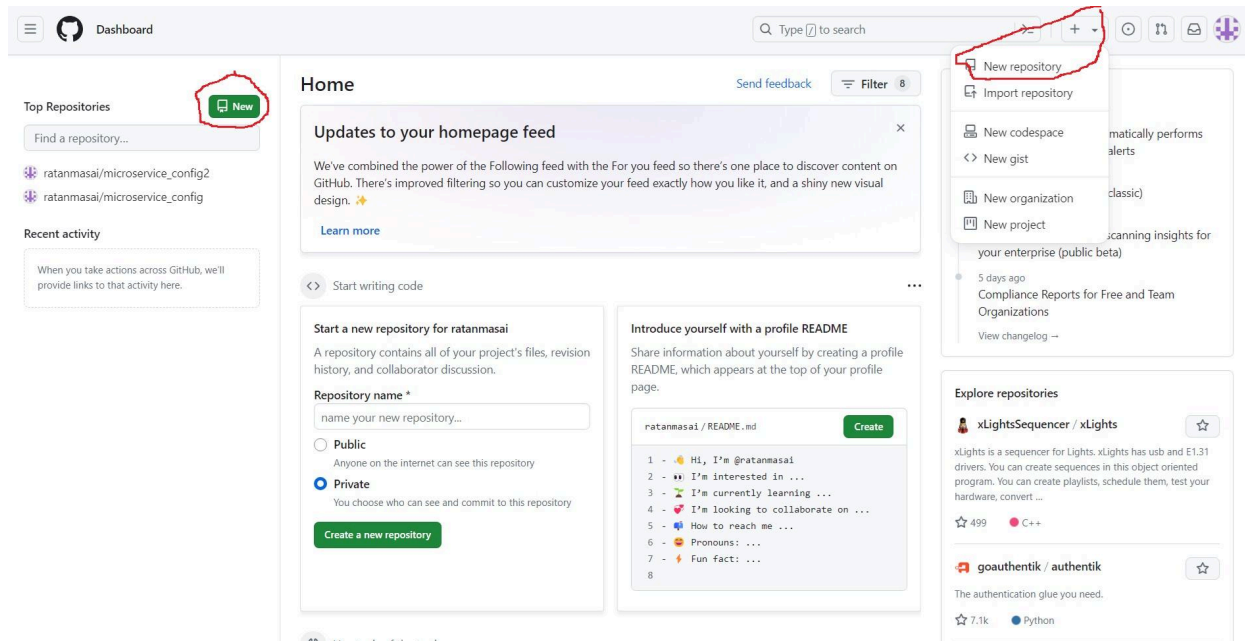
- Sign up with the GitHub
- Sign In GitHub

After successfully sign-in to the GitHub we will see the GitHub Dashboard:



Creating a new Repository inside the GitHub:

Inside the Dashboard at the left side we can see all are previously created remote repositories.



Give a valid name of the remote repository:

Required fields are marked with an asterisk (*).

Owner * Repository name *

ratanmasai sage_project1

✓ sage_project1 is available.

Great repository names are short and memorable. Need inspiration? How about [super-guide](#) ?

Description (optional)

☒ **Public**
Anyone on the internet can see this repository. You choose who can commit.

☐ **Private**
You choose who can see and commit to this repository.

Initialize this repository with:

☐ **Add a README file**
This is where you can write a long description for your project. [Learn more about READMEs.](#)

Add .gitignore

.gitignore template: None

Choose which files not to track from a list of templates. [Learn more about ignoring files.](#)

Choose a license

License: None

A license tells others what they can and can't do with your code. [Learn more about licenses.](#)

You are creating a public repository in your personal account.



Once we create a remote repository, we will see the following screen:

The screenshot shows the GitHub interface for a repository named 'sage_project1'. At the top, there are buttons for 'Pin', 'Unwatch' (1), 'Fork' (0), and 'Star' (0). Below this, there are two cards: 'Set up GitHub Copilot' and 'Add collaborators to this repository'. The main section is titled 'Quick setup — if you've done this kind of thing before'. It features a 'Set up in Desktop' button, an 'or' separator, and 'HTTPS' and 'SSH' tabs. The 'HTTPS' tab is selected, showing the URL 'https://github.com/ratanmasai/sage_project1.git' in a text box. Below this, there are two code blocks for command-line setup. The first block is titled '...or create a new repository on the command line' and contains the following commands:

```
echo "# sage_project1" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/ratanmasai/sage_project1.git
git push -u origin main
```

 The second block is titled '...or push an existing repository from the command line' and contains the following commands:

```
git remote add origin https://github.com/ratanmasai/sage_project1.git
git branch -M main
git push -u origin main
```

The URL of our remote repository: https://github.com/ratanmasai/sage_project1.git

By using this url only we can communicate with this remote repository.

Note:

1. The max allowed file size in Github public repository: 100MB
2. The max allowed repo size in Github public repository: 2GB

To work with the remote repository from our local computer like performing the **pull**, **push** and **fetch** operations, we need to get authenticated from GitHub remote server.

For this we need to make use of **Personal Access Token (PAT)**.

A Personal Access Token (PAT) is a secure way to authenticate with GitHub and other Git servers. PATs are used in place of passwords for Git operations over HTTPS and

provide a higher level of security and control.

PATs replace password authentication for Git operations (e.g., git push, git pull) over HTTPS. As of August 2021, GitHub deprecated password authentication for these operations.

Creating a PAT:

1. **Navigate to Settings:** Go to your GitHub account settings.
2. **Developer Settings:** Click on "Developer settings" in the left sidebar.
3. **Personal Access Tokens:** Select "Personal access tokens" and click "Generate new token."
4. **Configure Scopes and Expiration:** Choose the permissions (scopes) the token needs and set an expiration date.
5. **Generate and Store:** Click "Generate token," then copy and store the token securely. It will not be shown again.

Whenever we perform any push or pull operations from the remote repository to our local system, Git will ask for the authentication using username and password, use your GitHub username and PAT respectively(In place of our password).

Their authentication will be asked for the very first time.

Next time onwards these **PAT** will be cached inside our computer and it won't ask again and again.

We can check and clear our credentials using following ways:

For Windows Credential Manager:

- Open "Credential Manager" from the Control Panel.
- Look for your GitHub credentials and remove them.

For macOS Keychain:

- Open "Keychain Access" from Applications > Utilities.
- Search for "github.com" and remove any stored credentials.

To work with the remote repository we have 2 options:

Approach1: Create an Empty Remote Repository and Clone It

Steps:

1. Create an empty repository on GitHub.(**my-remote-project-repo**)
2. Clone the empty repository to your local machine using:
 - Inside the desktop create an empty folder(remote-demo1) and open this folder inside the vs-code.
 - Open the git-bash inside this folder in vs-code
 - Type the following command.
3. Make changes to your local repository (add files, commit changes, etc.).
 - Create a1.txt file with some lines and commit inside the local repository.
4. Push changes to the remote repository using:

git clone <remote-repository-url>

- Here **my-remote-project-repo** folder will be created inside the **remote-demo1** folder.
- Now move inside this **my-remote-project-repo** folder in git-bash.

git push origin <branch-name>

Example

Note: if it is asking for authentication, then generate and use the Personal Access Token.

5. Create one file called a2.txt using the remote repository i.e inside the GitHub and add some lines and then commit it inside the remote

repository.

6. Now to get this a2.txt file inside the local repository, we need to make use of the following command.

git pull

Advantages:

- **Simple Initialization:** GitHub provides a straightforward interface to create a new repository, making it easy to set up a new project.
- **Ready-to-go Setup:** Cloning the repository sets up the remote origin automatically, so you don't need to manually add the remote URL later.
- **Standard Practice:** This method is often recommended for starting new projects and is widely understood by most developers.

Disadvantages:

- **Initial Setup:** Requires internet access to clone the repository and set up the local environment.

Approach 2: Create a Local Repository and Push to a Remote Repository

Steps:

1. Create a local repository on your machine using:
 - Create a folder called my-remote-project-demo2 on the Desktop and open this folder inside the vs-code.
 - Initialize this folder as a git local repository by using the following command in git-bash.

git init

2. Add files and commit changes locally:

- Create a1.txt file with one line text.
- Add that file inside the staging area

git add a1.txt

- Commit the changes inside the local repository.

git commit -m "a1.txt file is created inside the local-repo and committed"

3. Create a new repository on GitHub without any content (do not initialize with README, .gitignore, etc.).
 - Here we can create the remote repository with any name, but it is recommended to create with the same name by which we created the local repository initially.

4. Add the remote repository to your local repository:

git remote add origin <remote-repository-url>

5. Rename the master branch to the main branch.

git branch -m master main

6. Push your local commits to the remote repository:

git push -u origin <branch-name>

Example:

git push -u origin main

Note: we can create a new branch locally, do some modification and push that branch only to the remote repository.

git push -u origin <branch-name>

Advantages:

- **Offline Work:** Allows you to set up and work on your repository without needing immediate internet access.
- **Flexibility:** Provides flexibility in setting up the repository structure before pushing it to a remote location.

Disadvantages:

- **Manual Setup:** Requires additional steps to add the remote URL and push the initial commits, which might be slightly more complex for beginners.

Recommendation:

Both approaches are valid, and the best choice depends on your workflow and project needs:

- **For Beginners and Standard Workflows:** Approach 1 is often recommended. It's simpler and ensures that the remote and local repositories are correctly linked from the start.
- **For Offline Work and Advanced Users:** Approach 2 can be more flexible and allows for initial work to be done offline before pushing to a remote repository.

Additional Considerations:

- **Collaboration:** If you're working in a team, cloning an empty repository (Approach 1) ensures that everyone starts with the same setup and avoids configuration mismatches.
- **Project Initialization:** If you need to set up a complex project structure or include certain initial files before linking to a remote repository, starting locally (Approach 2) might be beneficial.

Ultimately, both methods achieve the same result: a synchronized local and remote

repository. Choose the one that best fits your workflow and comfort level with Git.

Collaborators in GitHub:

Collaborators in GitHub are users who are granted access to a repository, allowing them to contribute to the project by committing code, managing issues, and performing other repository activities.

By managing collaborators effectively, you can ensure smooth and efficient collaboration on GitHub projects.

Roles and Permissions:

- **Write Access:** Collaborators typically have write access to the repository, meaning they can push commits, create branches, and manage issues and pull requests.
- **Admin Access:** Collaborators with admin access can manage repository settings, including adding or removing collaborators, and changing repository visibility.

Adding Collaborators:

1. **Navigate to the Repository:**
Go to the repository on GitHub where you want to add collaborators.
2. **Access Settings:**
Click on the "Settings" tab located at the top of the repository page.
3. **Manage Access:**
In the settings sidebar, click on "Manage access" or "Collaborators" under the "Access" section.
4. **Invite Collaborators:**

Click the "Invite a collaborator" button and enter the username, full name, or email of the person you want to add. Select the appropriate user from the dropdown list.

5. Set Permissions:

Specify the role (write or admin) and send the invitation. The invitee will receive a notification to accept the collaboration request.

Demo of Adding Collaborator:

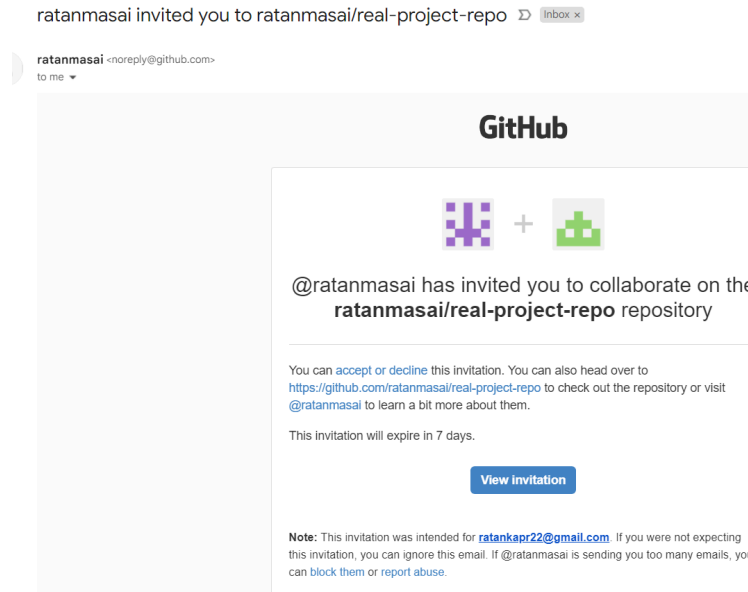
1. **Repository Owner:** [ratankaprgithub](#) creates a new repository called **real-project-repo** on GitHub by adding one **README** file.

2. Adding Collaborator:

- [ratankaprgithub](#) navigate to <https://github.com/ratankaprgithub/real-project-repo>
- He clicks on "Settings" > "Collaborators" > "Manage access" > "Add People".
- [ratankaprgithub](#) enters [ratankumargithub](#)(e.g., ratankumarkgr@gmail.com) and sends the invitations.

3. [ratankumargithub](#) accepts invitation:

- **ratankumargithub** receives an email notification about the invitation.
- He clicks on the invitation link and accepts it.



4. Collaborator Actions:

- **Clone the Repository:** ratankumargithub can now clone the repository to his local machine:

- Right click on the Desktop and open the git-bash terminal and paste the url.

git clone

`https://github.com/ratankaprgithub/real-project-repo.git`

- Here on the Desktop of **ratankumargithub** the remote repository created by **ratankaprgithub** is cloned and a new local-repository(new folder) is created with the name **real-project-repo**.

- **Configure the username and email for this specific repository**
Open this local repository folder inside the vs-code and inside the vs-code open the git-bash and type the following command to configure the username and email.

```
git config user.name "ratankumar"  
git config user.email "ratankumarkgr@gmail.com"
```

- **Create a Branch:** ratankumargithub creates a new branch for his work.

git switch -c feature-branch

- **Commit and Push Changes:** ratankumargithub commits his changes and pushes them to the remote repository:

- Create a new file a1.txt with the following line and commit this changes to the local-repositoy first.

a1.txt:

This is the first line added by the ratangithub

- Add this changes to the staging area and commit it to the local repository.

git add a1.txt

git commit -m "a1.txt created in feature branch and committed"

- Push this feature-branch to the remote repository.

git push origin feature-branch

- If it is asking for the authentication, then **ratankumargithub** needs to pass his username and Personal Access Token as his credentials.

- **Open a Pull Request:** ratankumargithub opens a pull request on GitHub to merge his changes into the main branch.

- Navigate to the repository on GitHub.
- Click on the "Pull requests" tab.
- Click "New pull request."
- Select the base branch (e.g., main) and the compare branch (e.g., feature-branch).
- Click "Create pull request."
- Add a descriptive title and description for the pull request.
- Click "Create pull request."

- **Accepting a Pull request:** [ratankaprgithub](#) can review and merge [ratankaprgithub](#)'s pull request by navigating to the "Pull Requests" tab, under the "**real-project-repo**" reviewing the changes, and clicking "Merge pull request" or he can give some feedback comment to the [ratankumargithub](#) .

Best Practices:

- **Use Pull Requests:** Encourage collaborators to use pull requests for all changes. This allows for code review and discussion before merging changes into the main branch.
- **Define Contribution Guidelines:** Create a CONTRIBUTING.md file in your repository to outline contribution rules and guidelines.
- **Regular Communication:** Use GitHub issues, pull request comments, and discussions to maintain clear communication among collaborators.

Pull Request / PR:

- A pull request (PR) in GitHub is a method for submitting contributions to a project. It allows developers to notify team members about changes they've made to a branch in a repository. Once a pull request is submitted, other team members can review the changes, discuss potential modifications, and merge the changes into the main codebase.

Key Components:

- **Branches:** Pull requests are typically made from a feature branch to the main branch (or another target branch).
- **Commits:** Pull requests include all commits made to the feature branch.
- **Reviewers:** Team members who review and discuss the changes proposed in the pull request.

- **Merge:** Integrating the changes from the pull request into the target branch.